CSE 102 - COMPUTER PROGRAMMING II
GENERİCS

Joseph LEDET
Department of Computer Engineering
Akdeniz University
josephledet@akdeniz.edu.tr

OBJECTİVES

- To know the benefits of generics (§21.1).
- To use generic classes and interfaces (§21.2).
- To declare generic classes and interfaces (§21.3).
- To understand why generic types can improve reliability and readability (§21.3).
- To declare and use generic methods and bounded generic types (§21.4).
- To use raw types for backward compatibility (§21.5).
- To know wildcard types and understand why they are necessary (§21.6).
- To convert legacy code using JDK 1.5 generics (§21.7).
- To understand that generic type information is erased by the compiler and all instances of a generic class share the same runtime class file (§21.8).
- To know certain restrictions on generic types caused by type erasure (§21.8).
- To design and implement generic matrix classes (§21.9).

WHY DO YOU GET A WARNİNG?

```
public class ShowUncheckedWarning {
  public static void main(String[] args) {
    java.util.ArrayList list =
      new java.util.ArrayList();
    list.add("Java Programming");
  }
}
```

To understand the compile warning on this line, you need to learn JDK 1.6 generics.

FİX THE WARNİNG

```
public class ShowUncheckedWarning {
  public static void main(String[] args) {
    java.util.ArrayList<String> list =
      new java.util.ArrayList<String>();
    list.add("Java Programming");
  }
}
```

No compile warning on this line.

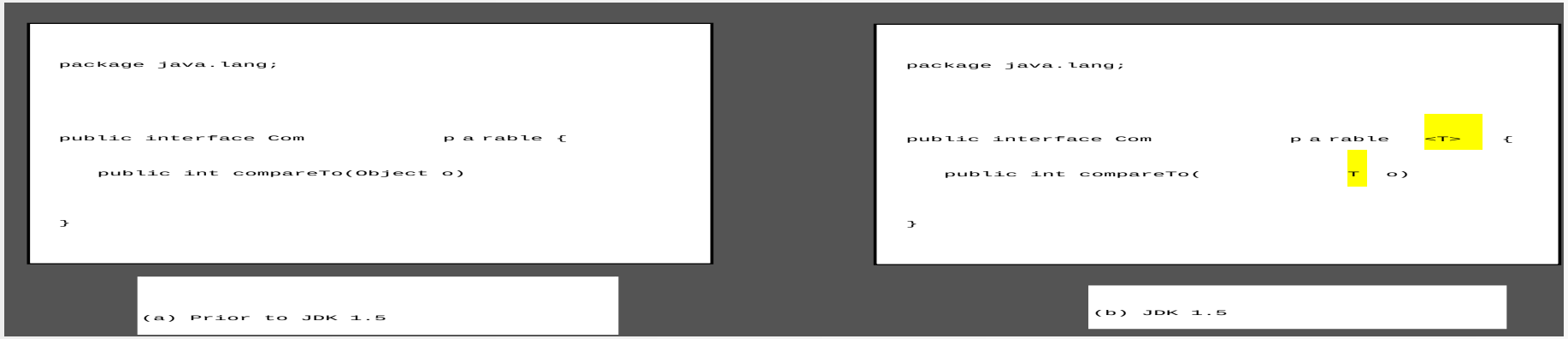
WHAT İS GENERİCS?

- *Generics* is the capability to parameterize types. With this capability, you can define a class or a method with generic types that can be substituted using concrete types by the compiler. For example, you may define a generic stack class that stores the elements of a generic type. From this generic class, you may create a stack object for holding strings and a stack object for holding numbers. Here, strings and numbers are concrete types that replace the generic type.

WHY GENERİCS?

- The key benefit of generics is to enable errors to be detected at compile time rather than at runtime. A generic class or method permits you to specify allowable types of objects that the class or method may work with. If you attempt to use the class or method with an incompatible object, a compile error occurs.

GENERİC TYPE



Generic Instantiation

Runtime error

```
Comparable c = new Date();
System.out.println(c.compareTo("red"));
```

(a) Prior to JDK 1.5

```
Comparable<Date> c = new Date();
System.out.println(c.compareTo("red"));
```

(b) JDK 1.5

Improves reliability

Compile error

GENERİC ARRAYLİST İN JDK 1.5

java.util.ArrayList
+ArrayList()
+add(o: Object) : void
+add(index: int, o: Object) : void
+clear(): void
+contains(o: Object): boolean
+get(index: int) : Object
+indexOf(o: Object) : int
+isEmpty(): boolean
+lastIndexOf(o: Object) : int
+remove(o: Object): boolean
+size(): int
+remove(index: int) : boolean
+set(index: int, o: Object) : Object

(a) ArrayList before JDK 1.5

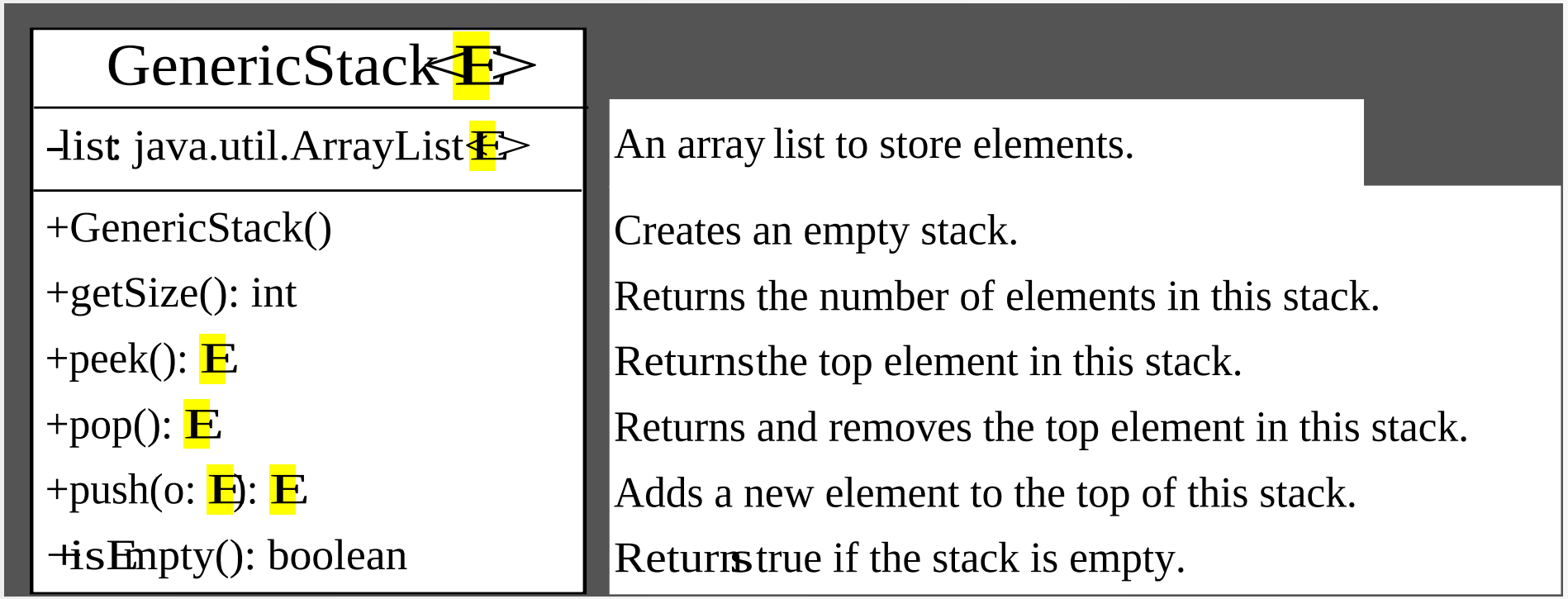
java.util.ArrayList<E>
+ArrayList()
+add(o: E) : void
+add(index: int, o: E) : void
+clear(): void
+contains(o: Object): boolean
+get(index: int) : E
+indexOf(o: Object) : int
+isEmpty(): boolean
+lastIndexOf(o: Object) : int
+remove(o: Object): boolean
+size(): int
+remove(index: int) : boolean
+set(index: int, o: E) : E

(b) ArrayList in JDK 1.5

NO CASTİNG NEEDED

```
ArrayList<Double> list = new ArrayList<Double>();
list.add(5.5); // 5.5 is automatically converted to new Double(5.5)
list.add(3.0); // 3.0 is automatically converted to new Double(3.0)
Double doubleObject = list.get(0); // No casting is needed
double d = list.get(1); // Automatically converted to double
```

DECLARING GENERIC CLASSES AND INTERFACES



GenericStack

GENERİC METHODS

```
public static <E> void print(E[] list) {
  for (int i = 0; i < list.length; i++)
    System.out.print(list[i] + " ");
  System.out.println();
 }
```

```
public static void print(Object[] list) {
  for (int i = 0; i < list.length; i++)
    System.out.print(list[i] + " ");
  System.out.println();
 }
```

BOUNDED GENERİC TYPE

```
public static void main(String[] args ) {
    Rectangle rectangle = new Rectangle(2, 2);
    Circle9 circle = new Circle9(2);
    System.out.println("Same area? " + equalArea(rectangle, circle));
}

public static <E extends GeometricObject> boolean
        equalArea(E object1, E object2) {
    return object1.getArea() == object2.getArea();
}
```

RAW TYPE AND BACKWARD COMPATİBİLİTY

```
// raw type
ArrayList list = new ArrayList();
```

This is *roughly* equivalent to

```
ArrayList<Object> list = new ArrayList<Object>();
```

RAW TYPE İS UNSAFE

```
// Max.java: Find a maximum object
public class Max {
  /** Return the maximum between two objects */
  public static Comparable max(Comparable o1, Comparable o2) {
    if (o1.compareTo(o2) > 0)
      return o1;
    else
      return o2;
  }
}
```

Runtime Error:

```
Max.max("Welcome", 23);
```

MAKE İT SAFE

```
// Max1.java: Find a maximum object
public class Max1 {
  /** Return the maximum between two objects */
  public static <E extends Comparable<E>> E max(E o1, E o2) {
    if (o1.compareTo(o2) > 0)
      return o1;
    else
      return o2;
  }
}
```

```
Max.max("Welcome", 23);
```

WİLDCARDS

Why wildcards are necessary? See this example.

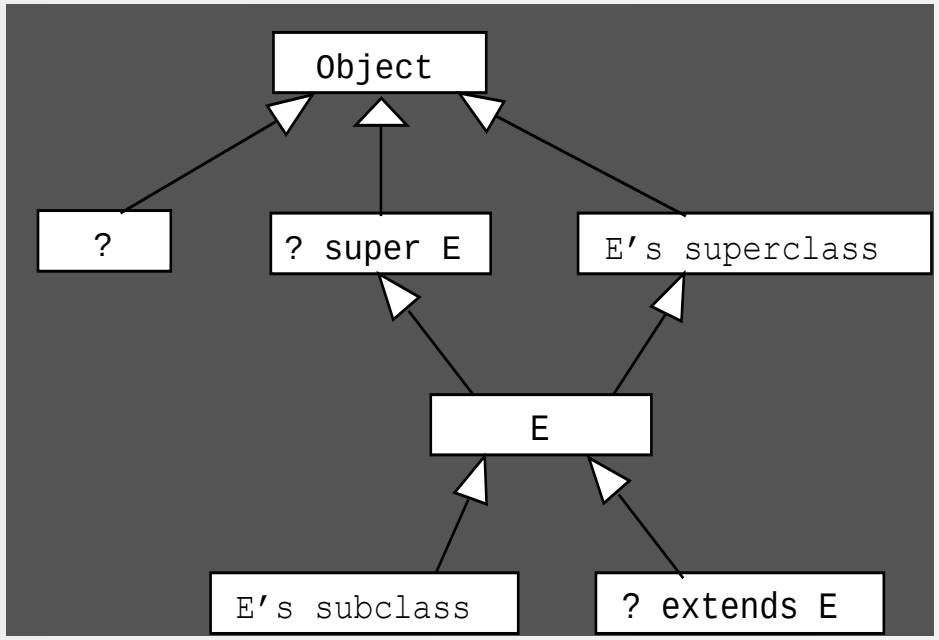
WildCardNeedDemo

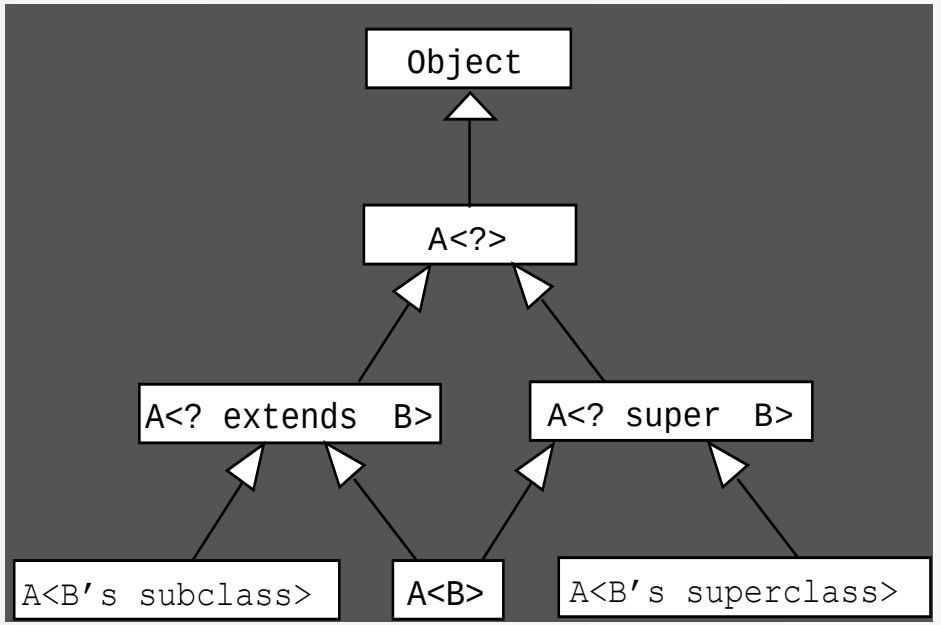
?	unbounded wildcard
? extends T	bounded wildcard
? super T	lower bound wildcard

AnyWildCardDemo

SuperWildCardDemo

GENERİC TYPES AND WİLDCARD TYPES





AVOİDİNG UNSAFE RAW TYPES

Use

new ArrayList<ConcreteType>()

Instead of

new ArrayList();

ERASURE AND RESTRICTIONS ON GENERİCS

Generics are implemented using an approach called *type erasure*. The compiler uses the generic type information to compile the code, but erases it afterwards. So the generic information is not available at run time. This approach enables the generic code to be backward-compatible with the legacy code that uses raw types.

COMPİLE TİME CHECKİNG

For example, the compiler checks whether generics is used correctly for the following code in (a) and translates it into the equivalent code in (b) for runtime use. The code in (b) uses the raw type.

```
ArrayList<String> list = new ArrayList<String>();
list.add("Oklahoma");
String state = list.get(0);
```

(a)

```
ArrayList list = new ArrayList();
list.add("Oklahoma");
String state = (String)(list.get(0));
```

(b)

IMPORTANT FACTS

It is important to note that a generic class is shared by all its instances regardless of its actual generic type.

```
GenericStack<String> stack1 = new
  GenericStack<String>();
GenericStack<Integer> stack2 = new
  GenericStack<Integer>();
```

Although GenericStack<String> and GenericStack<Integer> are two types, but there is only one class GenericStack loaded into the JVM.

RESTRİCTİONS ON GENERİCS

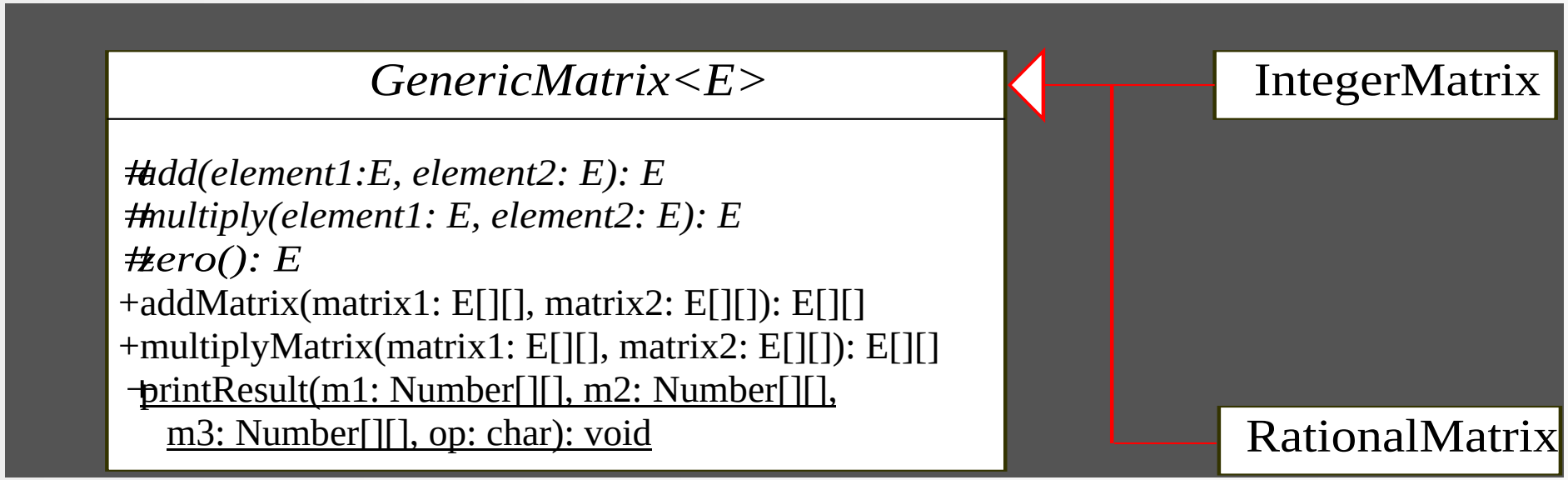
- Restriction 1: Cannot Create an Instance of a Generic Type. (i.e., new E()).
- Restriction 2: Generic Array Creation is Not Allowed. (i.e., new E[100]).
- Restriction 3: A Generic Type Parameter of a Class Is Not Allowed in a Static Context.
- Restriction 4: Exception Classes Cannot be Generic.

DESİGNİNG GENERİC MATRİX CLASSES

- Objective: This example gives a generic class for matrix arithmetic. This class implements matrix addition and multiplication common for all types of matrices.

GenericMatrix

UML DİAGRAM


GenericMatrix<E>
#add(element1:E, element2: E): E
#multiply(element1: E, element2: E): E
#zero(): E
+addMatrix(matrix1: E[][], matrix2: E[][]): E[][]
+multiplyMatrix(matrix1: E[][], matrix2: E[][]): E[][]
+printResult(m1: Number[][], m2: Number[][],
m3: Number[][], op: char): void
IntegerMatrix
RationalMatrix

SOURCE CODE

- Objective: This example gives two programs that utilize the GenericMatrix class for integer matrix arithmetic and rational matrix arithmetic.